Experiment – 1

Comparative Study of Low-Code Development versus Traditional Application Development

Aim

To examine low-code and traditional application development side by side, focusing on their development stages, workflow style, and the underlying technical trade-offs of each approach.

Introduction to Traditional Approach

Traditional development relies on writing every piece of application logic directly in code, using conventional editors, compilers, and IDEs. Engineers design database schemas, wire up APIs, and manage deployment infrastructure themselves, drawing on strong command of languages such as Java, Python, or C++. This hands-on approach grants complete authority over the codebase and system architecture, though it typically demands longer build times, larger budgets, and thorough testing cycles.

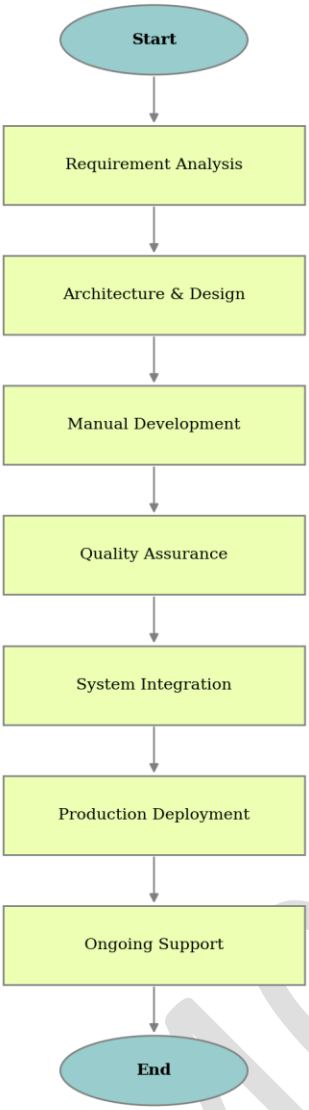
Introduction to Low-Code Approach

Low-Code Application Platforms replace much of the manual coding effort with cloud-hosted, visual builders. Developers assemble applications through drag-and-drop components, ready-made templates, and automated release pipelines, which lowers the barrier enough for "citizen developers" with little programming background to ship working software. By automating routine programming work, low-code speeds up delivery and trims upfront cost, at the expense of deep customization and some dependency on the chosen platform.

Stages in Traditional Approach

The traditional Software Development Life Cycle moves through a structured, engineering-heavy sequence, with dedicated effort required at every stage:

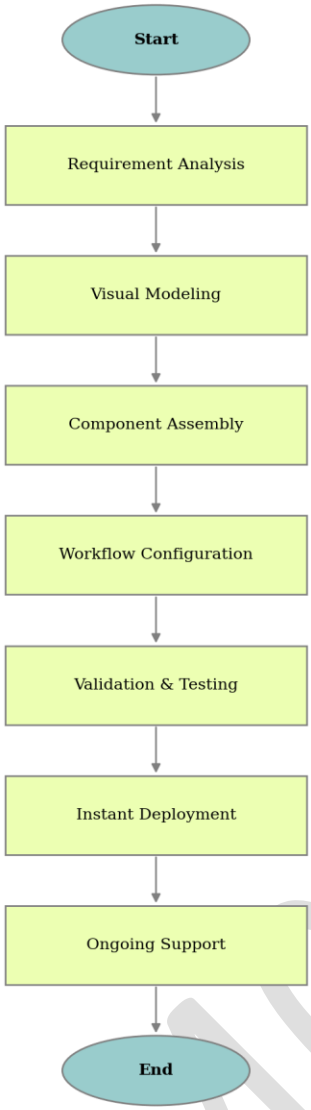
AD3153



2407157

Stages in Low-Code Approach

The low-code workflow swaps out manual coding stages for configuration-driven steps, each needing far less hands-on engineering:



2407157

Comparative Analysis

Feature	Traditional Development	Low-Code Development
Development Speed	Slower, as every module is hand-built and individually tested	Considerably faster, since ready-made components cut build time sharply
Skill Required	Demands trained developers fluent in programming languages and frameworks	Minimal coding skill needed; drag-and-drop tools suit citizen developers

Customization	Unrestricted — every line of logic can be shaped as needed	Bounded by the platform's available components and connectors
Cost	Larger upfront investment in developer time and infrastructure	Lower initial spend, offset by ongoing subscription or licensing fees
Maintenance & Updates	Every fix or enhancement requires manual code changes and re-testing	Platform vendor pushes updates centrally with little manual effort
Vendor Lock-in	Absent — the organization retains full ownership of the codebase	Present — the application is tied to the platform's ecosystem
Best Use Cases	Large-scale, performance-critical, or security-sensitive systems	Rapid prototyping, internal tools, and workflow automation

Result

The comparison confirms that traditional development remains the stronger choice for complex, high-performance, or tightly regulated systems, where full control matters most. Low-code, on the other hand, offers a quicker and more economical route for everyday business applications. Many organizations now blend the two — leaning on low-code for front-end and workflow needs while reserving traditional development for demanding back-end requirements.

240701573

ROLL NO : 240701573

NAME : UMAR FAROOK K